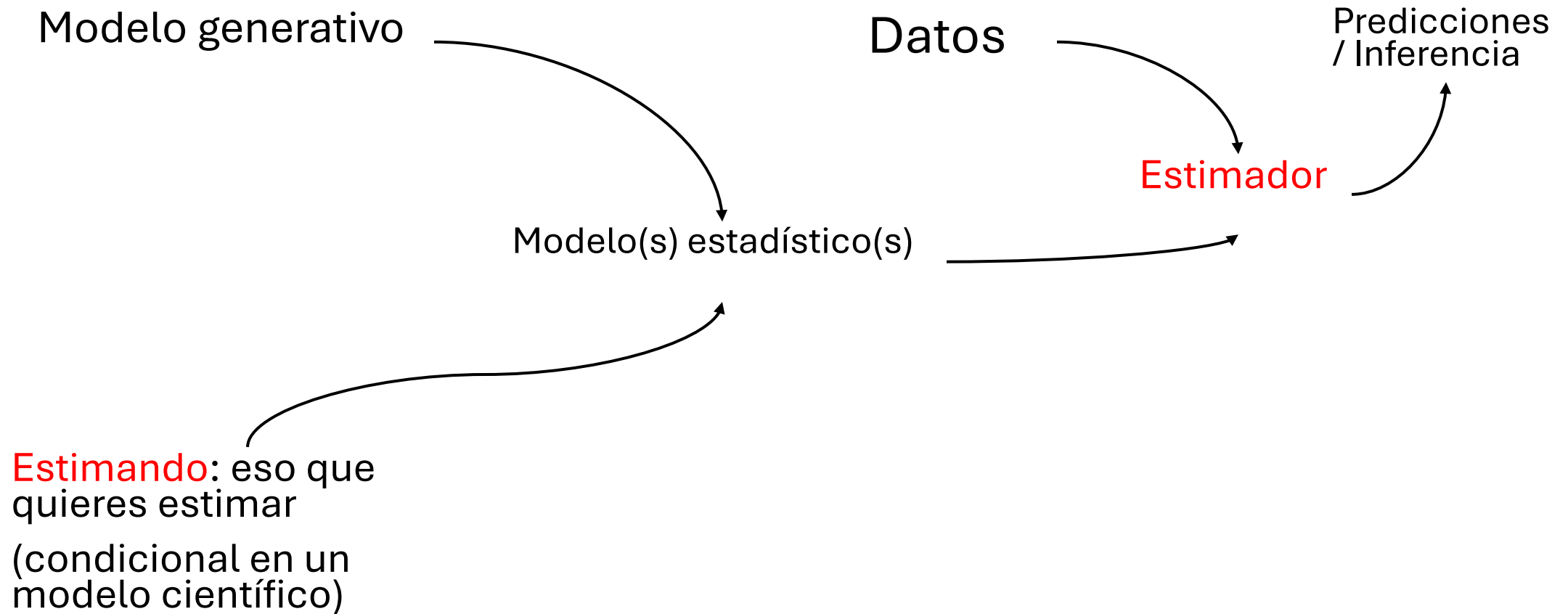


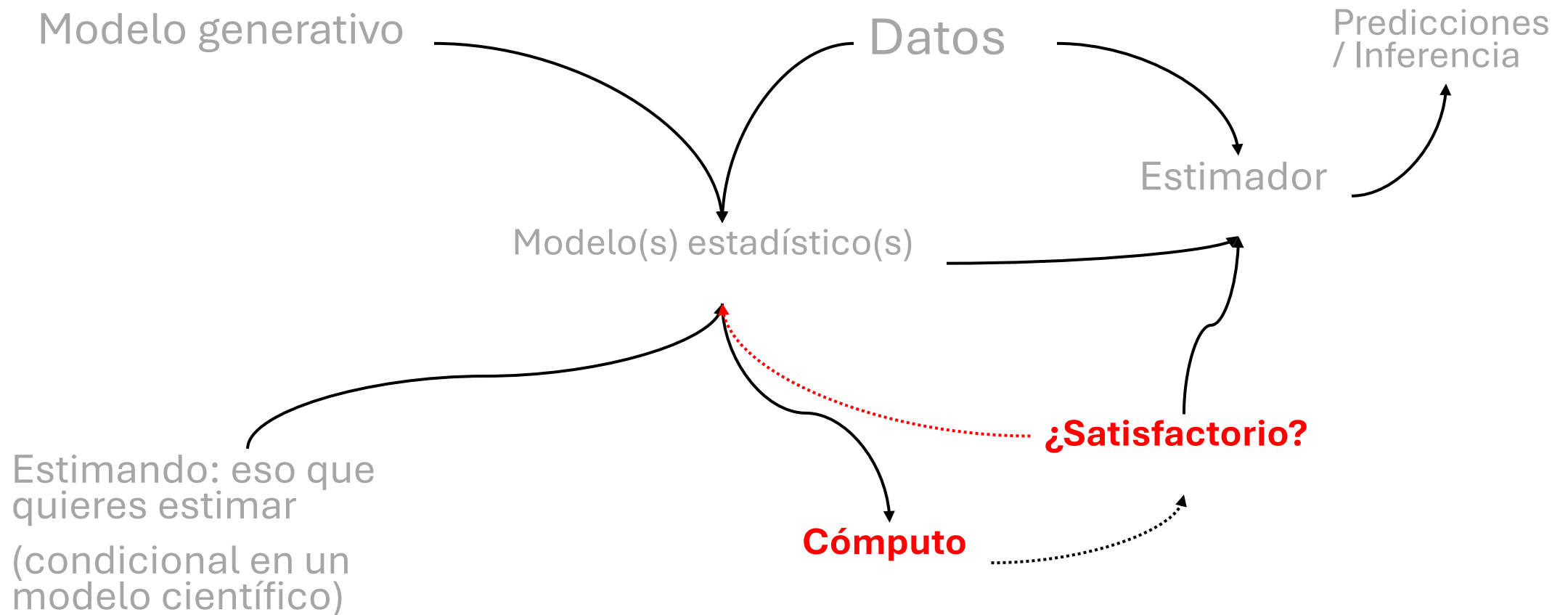
Estimación con Stan + brms Evaluación y comparación de modelos

Dr. Héctor Nájera
Dr. Curtis Huffman

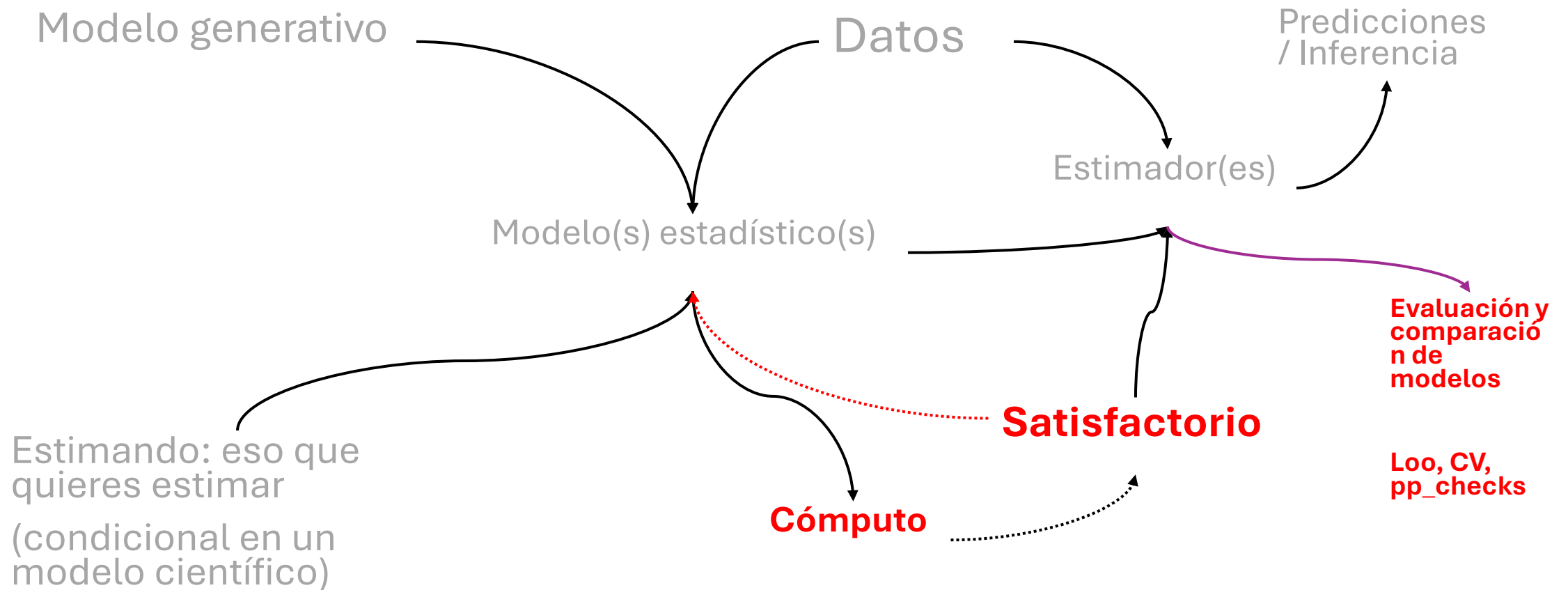
¿Dónde estamos?



¿De dónde venimos?



¿Dónde estamos?



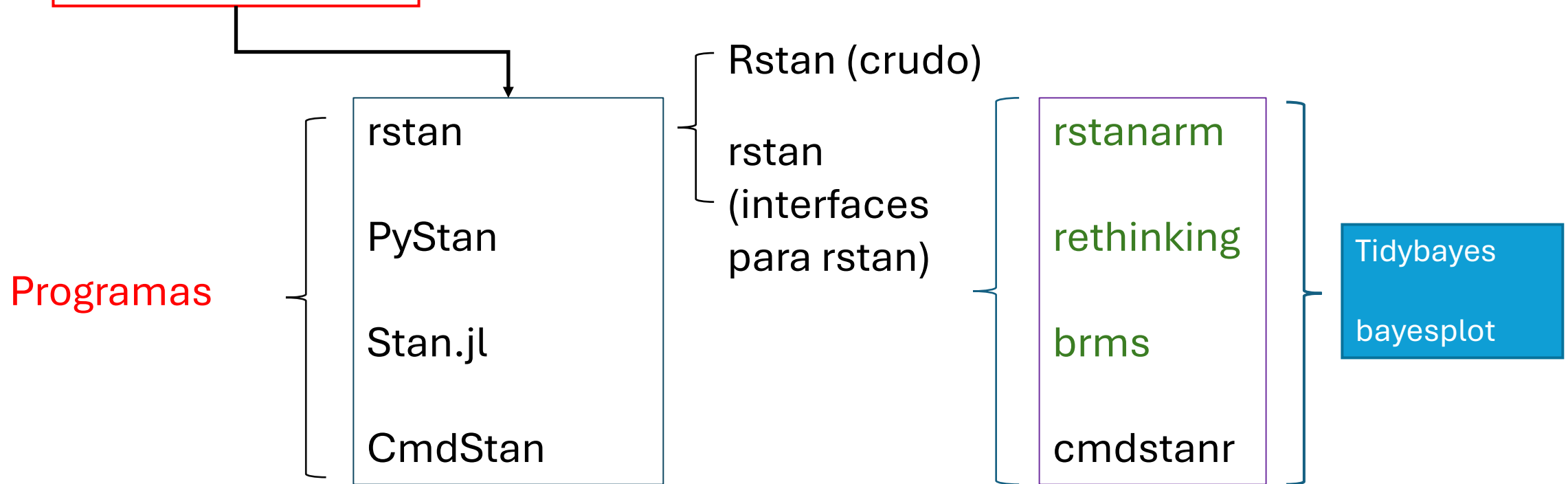
Stan: Diferentes plataformas



RStan

The R interface to Stan

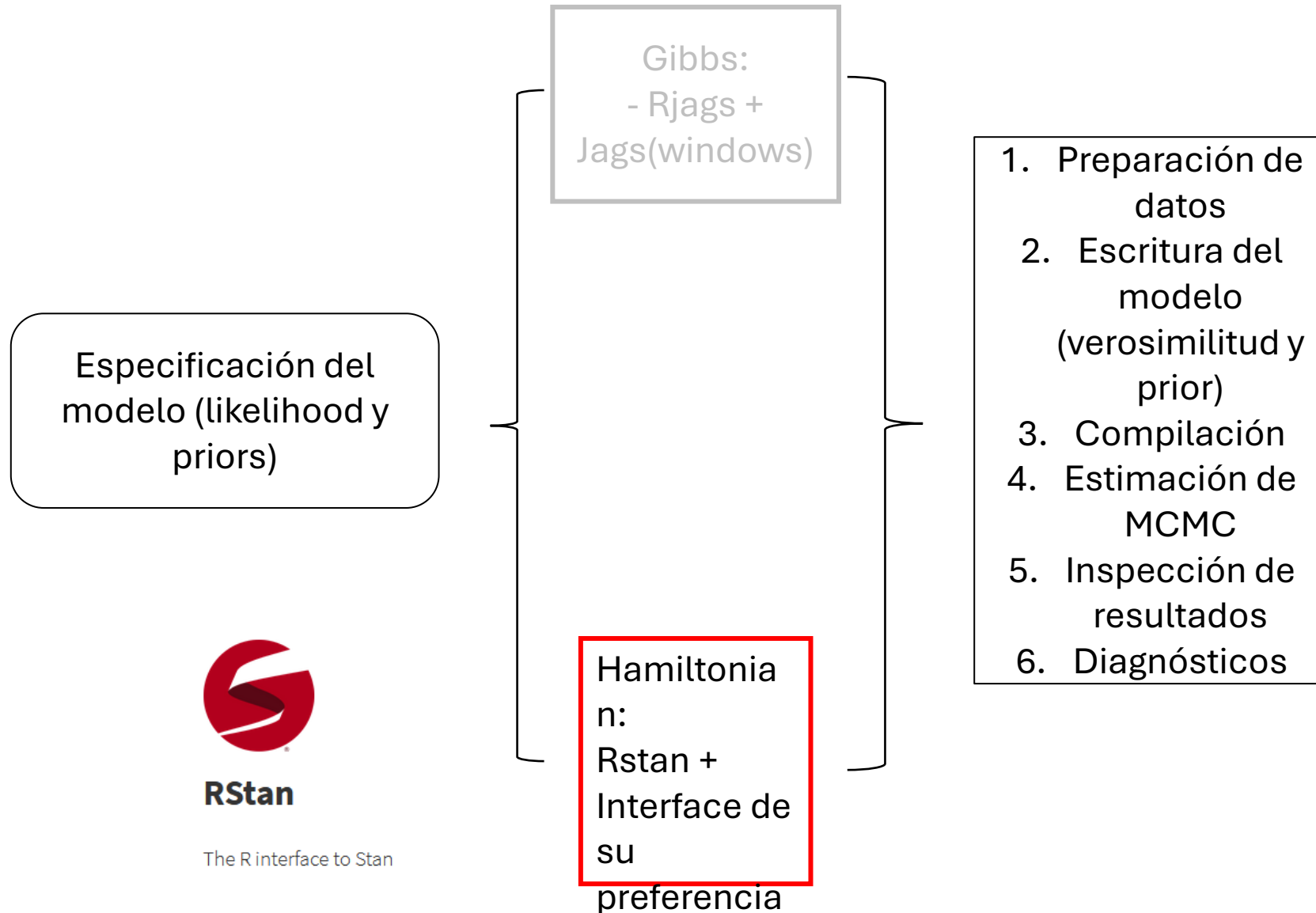
Stan está escrito
en C++



Avanzado: mayor potencia y
flexibilidad

Intermedio: menor flexibilidad

Pasos para la modelación



RStan

The R interface to Stan

Software

- Programa: R + Rstudio
- Algoritmo HMC: Rstan
- Interface: brms + cmdstanr



CmdStanR

(Non)linear multivariate, multilevel modeling via Stan

Software: JAX (PyTorch + TensorFlow)

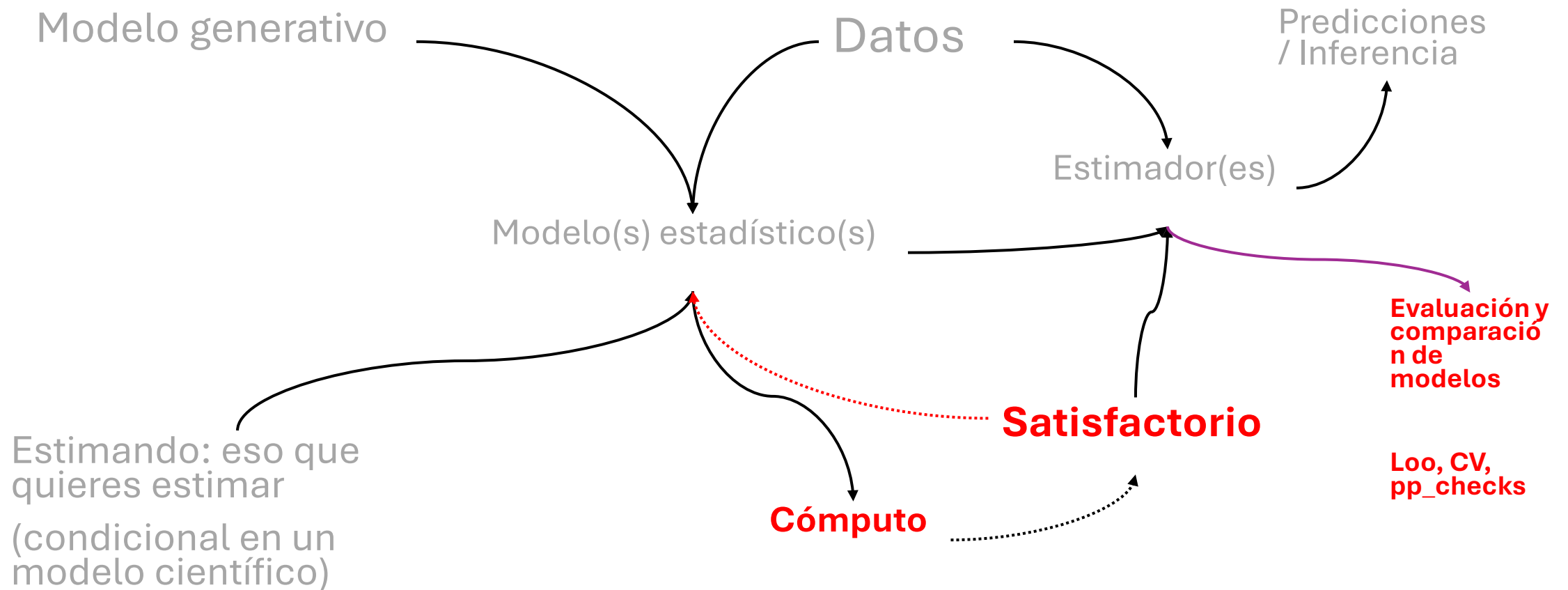
Motores de cómputo: **JAX** + PyTorch + TensorFlow

JAX: Diferenciación automatizada, crítica para el HMC-NUTS. Explota mejor las GPUs

Librerías probabilísticas: **NumPyro (JAX)** + Pyro (PyTorch)

En el futuro seguiremos usando (variantes) del HMC-NUTS pero probablemente en 10 años todos usemos JAX para modelos complejos. Por ahora, no tiene tanto sentido migrar a JAX debido a que toda la paquetería (y su documentación) está pensada para Rstan, PyStan y Julia

¿Dónde estamos?



¿Qué es un buen modelo?

- Capaz de hacer buenas predicciones dentro y fuera de la muestra
- Múltiples modelos: parámetros, variables, estructuras, distribuciones
- Comparación en los mismos términos dada esta heterogeneidad de opciones
- Tres aspectos: Loo-CV + WAIC + posterior predictive checks

brms

- `brm()` # Estimación del modelo
- `pp_check()` # Chequeo predictivo posterior o a priori
- `loo ()` # calculo de leave one out
- `loo_subsample()` # loo con 400 casos
- `loo_compare()` # comparación de múltiples modelos

modelo brms

```
M <- brm(
  pobreza ~ 1 +
    factor(educ) +
    tamhogesc +
    factor(loc),
  prior = c(
    set_prior(
      "normal(0,10)",
      class = "b")
    ),
  family = "bernoulli",
  chains = 4,
  iter = 2000
)
```

Fórmula

Define la variable outcome y los predictores. Igual que `lm()` — brms la traduce a Stan automáticamente.

prior =

Distribuciones a priori sobre los coeficientes. `class = "b"` cubre todos los β en un solo `set_prior()`.

family =

La verosimilitud del modelo. Bernoulli para binario, Gamma para positivos continuos, etc.

chains / iter

Control del muestreador MCMC. `chains` = cadenas independientes; `iter` = iteraciones totales por cadena.

`backend = "cmdstanr" + threads = threading(14) → paralelismo dentro de cada cadena`

- <https://github.com/hectornajera83/ClaseBayes2026>